CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 14: Unfolded RNN Architecture Visualization

Problem Statement: A new ML engineer at a speech-analytics firm finds the recurrent loop hard to follow. The team lead asks for a diagram showing the RNN unrolled over time. A 5-word input sentence passed through a SimpleRNN with a 3-unit hidden state. The objective is to visualise an RNN unfolded across time steps and trace the hidden-state values. Use TensorFlow, Keras, NetworkX, Matplotlib.

Tasks:

1. Build a SimpleRNN in Keras and extract its input, recurrent and output weights.
2. Compute hidden states for each time step manually and confirm them against Keras.
3. Draw the unfolded network as a graph with NetworkX (nodes x1...x5, h1...h5, y1...y5).
4. Label edges with shared weights W_xh, W_hh and W_hy.
5. Plot the hidden-state values across time steps.

FORMULA:

1. RNN Hidden State Calculation

$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$

2. Initial Hidden State

$h_0 = [0, 0, 0]$

3. Output Calculation

$y_t = W_y h_t + b_y$

4. Unfolded RNN Across Time Steps

$h_1 = \tanh(W_x x_1 + W_h h_0 + b_h)$

$h_2 = \tanh(W_x x_2 + W_h h_1 + b_h)$

$h_3 = \tanh(W_x x_3 + W h_2 + b)$

$h_4 = \tanh(W_x x_4 + W h_3 + b)$

$h_5 = \tanh(W_x x_5 + W h_4 + b)$

5. Shared Weights

W_x = Input-to-hidden weights

W = Hidden-to-hidden recurrent weights

W_y = Hidden-to-output weights

6. Hidden-State Mean

$\text{Mean} = (1/T) \sum_{t=1}^{T} h_t$

7. Hidden-State Change

$\Delta h_t = h_t - h_{t-1}$

8. Hidden-State Verification Error

$\text{Error} = |h_t(\text{manual}) - h_t(\text{Keras})|$

CODE:

```
import numpy as np
import tensorflow as tf
import networkx as nx
import matplotlib.pyplot as plt

# 1. Create a five-word input sentence
sentence = ["I", "love", "machine", "learning", "today"]

# Numerical representation of each word (5 time steps, 4 features)
X = np.array([
    [1.0, 0.0, 0.0, 0.0],
    [0.0, 1.0, 0.0, 0.0],
    [0.0, 0.0, 1.0, 0.0],
    [0.0, 0.0, 0.0, 1.0],
    [0.5, 0.5, 0.5, 0.5]
```

```
], dtype=np.float32)

X = X.reshape(1, 5, 4)

# 2. Build the SimpleRNN model
tf.random.set_seed(42)

inputs = tf.keras.Input(shape=(5, 4))
rnn_layer = tf.keras.layers.SimpleRNN(
    3, activation="tanh", return_sequences=True,
    name="simple_rnn"
)
hidden = rnn_layer(inputs)
output_layer = tf.keras.layers.Dense(1, name="output")
outputs = output_layer(hidden)

model = tf.keras.Model(inputs, outputs)

# Build the model
model(X)

# 3. Extract weights
Wxh, Whh, bh = rnn_layer.get_weights()
Why, by = output_layer.get_weights()

print("Input weights Wxh:\n", Wxh)
print("\nRecurrent weights Whh:\n", Whh)
print("\nHidden bias bh:\n", bh)
print("\nOutput weights Why:\n", Why)
print("\nOutput bias by:\n", by)

# 4. Calculate hidden states manually
h = np.zeros((1, 3), dtype=np.float32)
manual_states = []

for t in range(5):
    xt = X[:, t, :]
    h = np.tanh(xt @ Wxh + h @ Whh + bh)
    manual_states.append(h.copy())
```

```
manual_states = np.concatenate(manual_states, axis=0)

# Get hidden states from Keras
keras_states = rnn_layer(X).numpy()[0]

# Calculate outputs manually
manual_outputs = manual_states @ Why + by
keras_outputs = model(X).numpy()[0]

print("\nManual hidden states:\n", manual_states)
print("\nKeras hidden states:\n", keras_states)

print("\nManual outputs:\n", manual_outputs.flatten())
print("\nKeras outputs:\n", keras_outputs.flatten())

print("\nHidden states match:",
      np.allclose(manual_states, keras_states, atol=1e-6))
print("Outputs match:",
      np.allclose(manual_outputs, keras_outputs, atol=1e-6))

# 5. Draw the unfolded RNN using NetworkX
G = nx.DiGraph()

for t in range(1, 6):
    G.add_node(f"x{t}", layer="input")
    G.add_node(f"h{t}", layer="hidden")
    G.add_node(f"y{t}", layer="output")

    G.add_edge(f"x{t}", f"h{t}", label="W_xh")
    G.add_edge(f"h{t}", f"h{t}", label="W_hh")
    G.add_edge(f"h{t}", f"y{t}", label="W_hy")

    if t > 1:
        G.add_edge(f"h{t-1}", f"h{t}", label="W_hh")

# Use a custom layout for five time steps
pos = {}
for t in range(1, 6):
    pos[f"x{t}"] = (t, 2)
    pos[f"h{t}"] = (t, 1)
```

```
    pos[f"y{t}"] = (t, 0)

plt.figure(figsize=(14, 5))

node_colors = []
for node in G.nodes():
    if node.startswith("x"):
        node_colors.append("lightblue")
    elif node.startswith("h"):
        node_colors.append("lightgreen")
    else:
        node_colors.append("lightsalmon")

nx.draw_networkx_nodes(
    G, pos, node_size=1100, node_color=node_colors
)
nx.draw_networkx_labels(G, pos, font_size=10)

# Exclude self-loops from the normal edge drawing
normal_edges = [
    (u, v) for u, v in G.edges()
    if u != v
]
nx.draw_networkx_edges(
    G, pos, edgelist=normal_edges,
    arrows=True, arrowsize=18,
    connectionstyle="arc3,rad=0.08"
)

# Draw recurrent self-loops
nx.draw_networkx_edges(
    G, pos,
    edgelist=[(f"h{t}", f"h{t}") for t in range(1, 6)],
    arrows=True, arrowsize=18,
    connectionstyle="arc3,rad=0.45",
    min_source_margin=15, min_target_margin=15
)

edge_labels = {
    (u, v): d["label"]
```

```
    for u, v, d in G.edges(data=True)
}
nx.draw_networkx_edge_labels(
    G, pos, edge_labels=edge_labels, font_size=8,
    label_pos=0.5
)

plt.title("Unfolded SimpleRNN Architecture")
plt.axis("off")
plt.tight_layout()
plt.show()

# 6. Plot hidden-state values across time steps
plt.figure(figsize=(10, 5))

for unit in range(3):
    plt.plot(
        range(1, 6),
        manual_states[:, unit],
        marker="o",
        label=f"Hidden Unit {unit + 1}"
    )

plt.xticks(range(1, 6), sentence)
plt.xlabel("Time Step / Word")
plt.ylabel("Hidden State Value")
plt.title("Hidden-State Values Across Five Time Steps")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Input weights Wxh:
 [[ 0.77130365 -0.74450386  0.12503886]
 [ 0.71379113 -0.85987025  0.2336694 ]
 [-0.353243    0.8428689   0.6085243 ]
 [-0.893977    0.29113722  0.56819606]]

Recurrent weights Whh:
 [[ 0.80958223 -0.30325034 -0.5026093 ]
 [-0.23637621  0.61532205 -0.75200075]
```

```
 [ 0.537311    0.7276112   0.42647254]]

Hidden bias bh:
 [0. 0. 0.]

Output weights Why:
 [[0.5911319 ]
 [0.21041489]
 [0.7419075 ]]

Output bias by:
 [0.]

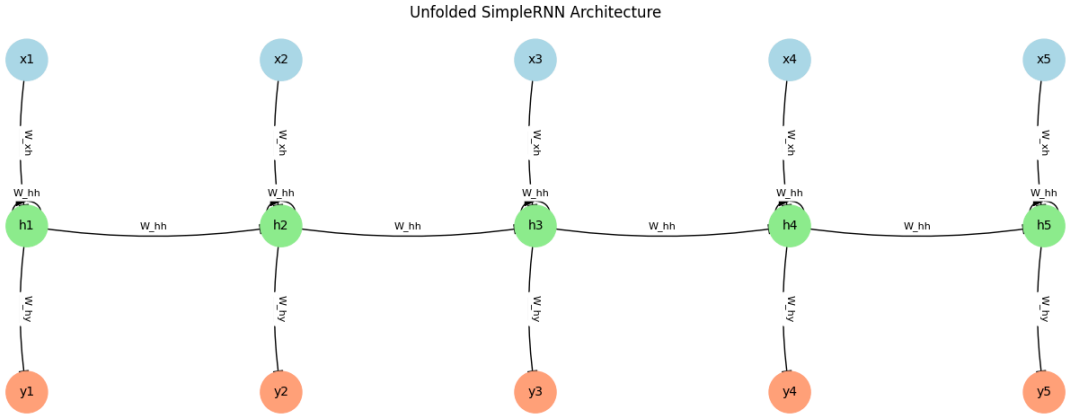
Manual hidden states:
 [[ 0.6476869  -0.6318586   0.12439126]
 [ 0.89654773 -0.8751276   0.410609  ]
 [ 0.66407603  0.31966043  0.75784   ]
 [-0.02471212  0.6846755   0.30700767]
 [ 0.10169571  0.39439008  0.37668318]]

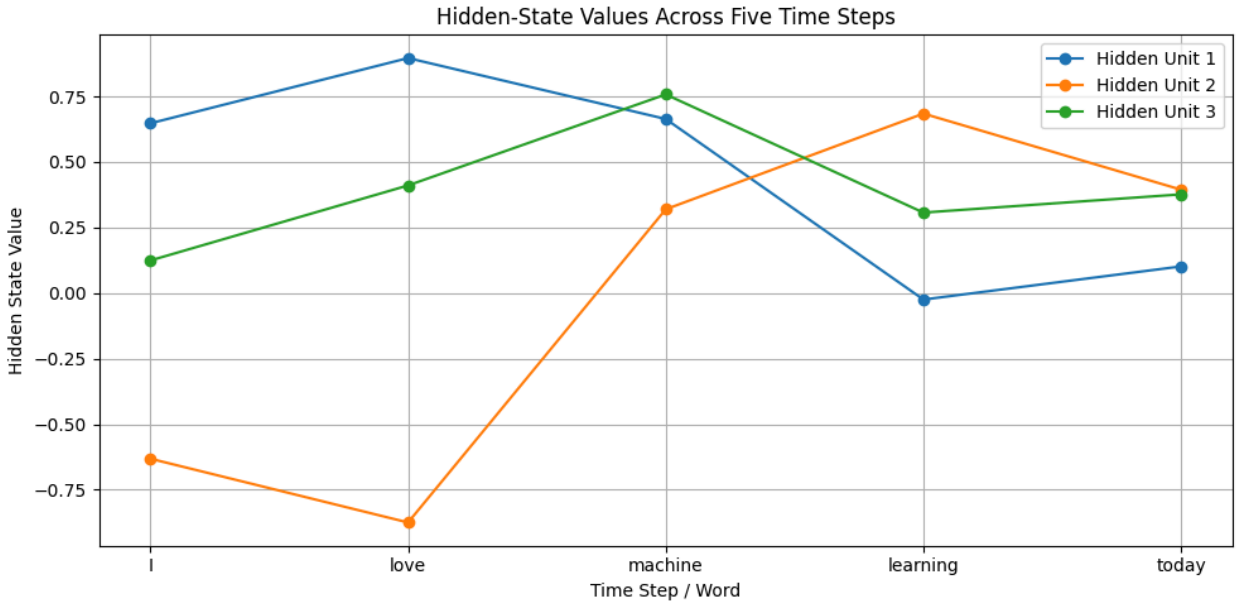
Keras hidden states:
 [[ 0.6476869  -0.63185865  0.12439125]
 [ 0.8965477  -0.8751276   0.41060904]
 [ 0.66407603  0.31966045  0.75784004]
 [-0.02471212  0.68467546  0.30700764]
 [ 0.10169573  0.39439002  0.37668315]]

Manual outputs:
 [0.34220275 0.650472   1.022065   0.35722908 0.4225652 ]

Keras outputs:
 [0.34220272 0.65047204 1.022065   0.35722905 0.42256516]

Hidden states match: True
Outputs match: True
```


Unfolded SimpleRNN Architecture
x1
x2
x3
x4
x5
W_xh
W_hh
h1
h2
h3
h4
h5
W_hy
y1
y2
y3
y4
y5


Hidden-State Values Across Five Time Steps
Hidden Unit 1
Hidden Unit 2
Hidden Unit 3
Hidden State Value
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
I
love
machine
learning
today
Time Step / Word